

Ảo giác AI (Hallucination) & Cơ chế Guardrails

Tài liệu giải thích hiện tượng ảo giác trong mô hình ngôn ngữ lớn (LLM), các loại ảo giác phổ biến, và hệ thống phòng thủ đa tầng (Guardrails) được triển khai trong DentalAI để đảm bảo độ tin cậy cho tư vấn y khoa.

Tham chiếu mã nguồn:

- `src/lib/constants.py` — 9 quy tắc cứng trong System Instructions
- `src/agent/chatbot.py` — Pipeline xử lý
- `src/retriever/search.py` — Overview Boost, Dynamic Weights

1. Ảo giác AI (Hallucination) là gì?

Ảo giác là hiện tượng LLM sinh ra nội dung **có vẻ đúng, trôi chảy về ngôn ngữ, nhưng thực tế sai hoặc bịa đặt** — không có cơ sở từ dữ liệu đầu vào.

Tại sao LLM bị ảo giác?

LLM hoạt động theo cơ chế **next-token prediction** — dự đoán từ tiếp theo dựa trên xác suất thống kê từ dữ liệu huấn luyện. Nó không "hiểu" hay "biết" theo nghĩa con người, mà chỉ sinh ra chuỗi token có xác suất cao nhất. Khi:

- Câu hỏi nằm ngoài phạm vi dữ liệu huấn luyện → LLM "lấp đầy khoảng trống" bằng thông tin có vẻ hợp lý
- Context không chứa đủ thông tin → LLM bổ sung từ kiến thức nội tại (parametric knowledge)
- Câu hỏi mơ hồ → LLM chọn diễn giải có xác suất cao nhất, có thể sai

Tại sao ảo giác nguy hiểm trong y khoa?

Trong chatbot giải trí, ảo giác chỉ gây khó chịu. Trong tư vấn y khoa:

- Thông tin sai về liều thuốc, quy trình điều trị → **nguy hiểm sức khỏe**
- Bịa tên thuốc, tên bệnh không tồn tại → mất uy tín hệ thống
- Trộn lẫn thông tin đúng và sai → khó phát hiện, nguy hiểm hơn sai hoàn toàn

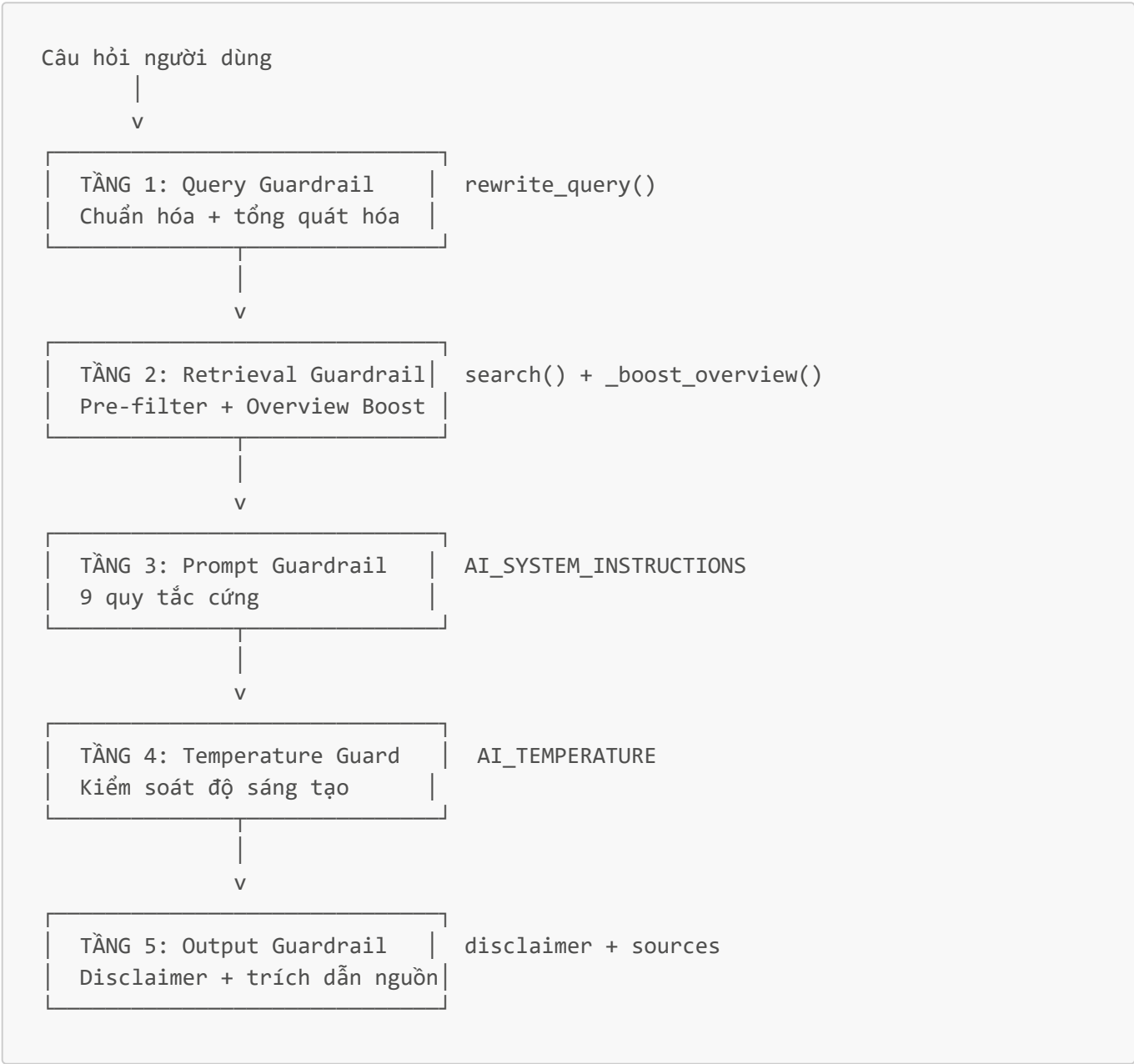
2. Phân loại ảo giác trong hệ thống RAG

Loại	Mô tả	Ví dụ
Intrinsic Hallucination	Mâu thuẫn với context đã cung cấp	Context: "niềng răng mất 12-24 tháng" → LLM trả lời "chỉ mất 3 tháng"
Extrinsic Hallucination	Thêm thông tin không có trong context	Context chỉ nói về sâu răng → LLM tự thêm thông tin về viêm nướu

Loại	Mô tả	Ví dụ
Detail Suggestion Bias	Lấy thông tin cụ thể của 1 thương hiệu trả lời cho câu hỏi chung	Hỏi "quy trình niềng răng" → trả lời quy trình riêng của Invisalign
Knowledge Overflow	Dùng kiến thức tự có thay vì context	Context không có thông tin → LLM tự trả lời từ dữ liệu huấn luyện

3. Kiến trúc Guardrails đa tầng

DentalAI triển khai **5 tầng phòng thủ** chống ảo giác, mỗi tầng hoạt động tại một giai đoạn khác nhau trong pipeline:



4. Chi tiết từng tầng Guardrail

Tầng 1: Query Guardrail — Chuẩn hóa đầu vào

File: `src/agent/chatbot.py` → `rewrite_query()`

Cơ chế	Mục đích	Ví dụ
Ghép ngữ cảnh từ lịch sử	Tránh query mơ hồ gây retrieval sai	"có đất không?" → "chi phí niềng răng"
Không thêm chi tiết cụ thể	Tránh LLM tự suy diễn thương hiệu/vị trí	Không thêm "Invisalign" nếu user không nhắc
Thêm "tổng quan" cho câu hỏi chung	Ưu tiên retrieval bài khái quát	"quy trình niềng răng" → "quy trình niềng răng tổng quan"

Đây là guardrail **phòng ngừa** — can thiệp trước khi tìm kiếm, đảm bảo retriever nhận query đúng ý đồ.

Tầng 2: Retrieval Guardrail — Kiểm soát nguồn dữ liệu

File: `src/retriever/search.py`

Category Pre-filter:

- `extract_category()` trích xuất bệnh/dịch vụ → thu hẹp từ 762 xuống ~60 tài liệu
- Giảm noise: bài về "sâu răng" không lọt vào context khi hỏi về "niềng răng"

Overview Boost:

- `_boost_overview()` đẩy bài có section "Tổng quan", "Giới thiệu", "Là gì" lên đầu
- Đảm bảo LLM nhìn thấy bài khái quát **trước** bài thương hiệu cụ thể
- Giảm thiểu Detail Suggestion Bias

Dynamic Weights:

- Câu hỏi keyword-heavy (chi phí, quy trình) → BM25 ưu tiên 0.7 → khớp chính xác từ khóa
- Câu hỏi semantic → cân bằng 0.5/0.5 → tránh bias từ kênh nào

Tầng 3: Prompt Guardrail — 9 quy tắc cứng

File: `src/lib/constants.py` → `AI_SYSTEM_INSTRUCTIONS`

Đây là tầng phòng thủ **mạnh nhất**, can thiệp trực tiếp vào hành vi sinh text của LLM:

Rule 2 — Strict Grounding (Chống Knowledge Overflow):

"CHỈ ĐƯỢC PHÉP trả lời dựa trên ngữ cảnh do hệ thống cung cấp"

Tác dụng: Cấm LLM dùng kiến thức tự có. Nếu context không chứa thông tin → không được bịa.

Rule 3 — Từ chối cố định (Chống Extrinsic Hallucination):

"Bạn **BẮT BUỘC** phải nói đúng 1 câu: Hiện tại kho dữ liệu của hệ thống chưa có thông tin cụ thể về vấn đề này..."

Tác dụng: Câu từ chối được **hard-code** — LLM không có quyền sáng tạo câu từ chối, tránh trường hợp nó "cố gắng giúp" bằng cách bịa thông tin.

Rule 4 — Giới hạn domain (Chống Off-topic Hallucination):

"CHỈ trả lời câu hỏi nha khoa. Từ chối lịch sự nếu ngoài phạm vi."

Tác dụng: Ngăn LLM trả lời câu hỏi ngoài chuyên môn (lập trình, nấu ăn,...) dù có thể tự tin trả lời đúng — vì ngoài phạm vi kiểm chứng của hệ thống.

Rule 9 — Chống ám thị chi tiết (Chống Detail Suggestion Bias):

"KHÔNG ĐƯỢC lấy quy trình của một thương hiệu cụ thể để trả lời cho câu hỏi chung"

"Nếu ngữ cảnh chỉ có thông tin của một hãng, phải nói rõ: Theo quy trình của [Tên Hãng]..."

Tác dụng: Ngăn LLM trình bày thông tin cụ thể của Invisalign/Straumann như thông tin chung của ngành. Bắt buộc ghi rõ nguồn nếu chỉ có 1 thương hiệu.

Tầng 4: Temperature Guard — Kiểm soát entropy

File: `src/lib/constants.py` → `AI_TEMPERATURE`

Temperature	Giá trị	LLM call	Hiệu ứng
STRICT	0.0	Rewrite, Extract	Deterministic, không sáng tạo → không bịa
NORMAL	0.3	Answer Stream	Đủ tự nhiên, vẫn bám sát context
0.5	(inline)	Expand Queries	Cần đa dạng từ đồng nghĩa

Temperature = 0.0 có nghĩa LLM **luôn chọn token có xác suất cao nhất** → loại bỏ hoàn toàn tính ngẫu nhiên → giảm ảo giác cho các tác vụ cần chính xác tuyệt đối.

Temperature = 0.3 cho Answer Stream là sự đánh đổi: đủ thấp để bám sát context, đủ cao để câu văn tự nhiên không như robot.

Tầng 5: Output Guardrail — Minh bạch đầu ra

File: `src/agent/chatbot.py` → `answer_stream()`, `src/chat/router.py`

Disclaimer tự động:

```
disclaimer = "\n\n*Thông tin chỉ mang tính tham khảo, không thay thế tư vấn trực tiếp từ bác sĩ nha khoa.*"
yield disclaimer
```

- Disclaimer được **code tự thêm**, không phải LLM sinh → đảm bảo luôn xuất hiện
- Rule 8 cấm LLM tự thêm disclaimer → tránh trùng lặp

Trích dẫn nguồn:

```
yield {
  "sources": retrieved_docs,
  "rewritten_query": rewritten_question
}
```

- Mỗi câu trả lời đều kèm danh sách tài liệu tham khảo (title, section, source URL)
- Người dùng có thể **kiểm chứng** thông tin bằng cách click vào nguồn gốc
- Đây là cơ chế **traceability** — mọi thông tin đều có thể truy vết

5. Ma trận phòng thủ

Bảng tổng hợp: mỗi loại ảo giác được phòng thủ bởi tầng nào.

Loại ảo giác	Tầng 1 (Query)	Tầng 2 (Retrieval)	Tầng 3 (Prompt)	Tầng 4 (Temp)	Tầng 5 (Output)
Knowledge Overflow			Rule 2, Rule 3	Temp = 0.3	
Extrinsic Hallucination		Category filter	Rule 2, Rule 3	Temp = 0.3	Sources
Detail Suggestion Bias	Thêm "tổng quan"	Overview Boost	Rule 9		
Off-topic			Rule 4		Disclaimer
Intrinsic Hallucination			Rule 2	Temp = 0.3	Sources
Mơ hồ do follow-up	Rewrite query				

6. Ví dụ thực tế: Phòng thủ hoạt động

Tình huống 1: Câu hỏi ngoài domain

User: "Cách nấu phở bò?"

Tầng 3 (Rule 4): Nhận diện ngoài phạm vi nha khoa
→ LLM từ chối lịch sự, không trả lời

Tình huống 2: Context không có thông tin

User: "Kem đánh răng Sensodyne có tốt không?"
Retriever: không tìm thấy bài về Sensodyne

Tầng 3 (Rule 3): Không có thông tin trong context
→ LLM trả đúng câu cố định: "Hiện tại kho dữ liệu của hệ thống chưa có thông tin cụ thể về vấn đề này..."
→ KHÔNG bịa review về Sensodyne dù LLM biết sản phẩm này

Tình huống 3: Âm thị thương hiệu

User: "Quy trình niềng răng như thế nào?"
Context: chứa 7/10 bài về Invisalign, 3/10 bài tổng quan

Tầng 1: Rewrite thêm "tổng quan" → ưu tiên bài khái quát
Tầng 2: Overview Boost đẩy bài "Niềng răng - Tổng quan" lên đầu
Tầng 3 (Rule 9): Nếu vẫn dùng thông tin Invisalign → phải ghi rõ nguồn
→ LLM trả lời quy trình chung, không trình bày như quy trình Invisalign

Tình huống 4: Câu hỏi follow-up mơ hồ

Lịch sử: User hỏi "niềng răng là gì?" → Bot trả lời
User: "mất bao lâu?"

Tầng 1: Rewrite "mất bao lâu?" → "thời gian niềng răng mất bao lâu"
→ Retriever tìm đúng bài về thời gian niềng răng
→ Không bị hiểu sai thành "nhổ răng mất bao lâu"

7. Giới hạn và hướng phát triển

Giới hạn hiện tại

Giới hạn	Giải thích
Prompt-based, không phải model-based	Guardrails hoạt động qua prompt instruction, LLM vẫn có thể vi phạm dù xác suất thấp

Giới hạn	Giải thích
Không có fact-checking tự động	Hệ thống tin tưởng LLM tuân thủ rules, không verify output so với context
Không có content filter	Không phát hiện nội dung nhạy cảm / nguy hiểm trong output

Hướng phát triển

Cải tiến	Mô tả
Output Validator	Thêm LLM call thứ 5 kiểm tra: "Câu trả lời này có bám sát context không?"
Confidence Score	Đánh giá mức độ tin cậy dựa trên overlap giữa output và context
Citation Verification	Tự động highlight phần nào trong câu trả lời đến từ tài liệu nào
Guardrail Model	Fine-tune model nhỏ chuyên phát hiện hallucination, chạy song song